

Think Java

How to Think Like a Computer Scientist

2nd Edition, Version 7.1.0

Think Java

How to Think Like a Computer Scientist

2nd Edition, Version 7.1.0

Allen B. Downey and Chris Mayfield

Green Tea Press

Needham, Massachusetts

Copyright © 2020 Allen B. Downey and Chris Mayfield.

Green Tea Press
9 Washburn Ave
Needham, MA 02492

Permission is granted to copy, distribute, and/or modify this work under the terms of the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License, which is available at <https://creativecommons.org/licenses/by-nc-sa/4.0/>.

The original form of this book is L^AT_EX source code. Compiling this code has the effect of generating a device-independent representation of the book, which can be converted to other formats and printed.

The L^AT_EX source for this book is available from <https://thinkjava.org/> and <https://github.com/ChrisMayfield/ThinkJava2>.

Contents

Preface

Think Java is an introduction to computer science and programming intended for readers with little or no experience. We start with the most basic concepts and are careful to define all terms when they are first used. The book presents each new idea in a logical progression. Larger topics, like control flow statements and object-oriented programming, are divided into smaller examples and introduced over the course of several chapters.

This book is intentionally concise. Each chapter is 12–14 pages and covers the material for one week of a college course. It is not meant to be a comprehensive presentation of Java, but rather, an initial exposure to programming constructs and techniques. We begin with small problems and basic algorithms and work up to object-oriented design. In the vocabulary of computer science pedagogy, this book uses the “objects late” approach.

The Philosophy Behind the Book

Here are the guiding principles that make the book the way it is:

One concept at a time: We break down topics that give beginners trouble into a series of small steps, so that they can exercise each new concept in isolation before continuing.

Balance of Java and concepts: The book is not primarily about Java; it uses code examples to demonstrate computer science. Most chapters start with language features and end with concepts.

Conciseness: An important goal of the book is to be small enough so that students can read and understand the entire text in a one-semester college or AP course.

Emphasis on vocabulary: We try to introduce the minimum number of terms and define them carefully when they are first used. We also organize them in glossaries at the end of each chapter.

Program development: There are many strategies for writing programs, including bottom-up, top-down, and others. We demonstrate multiple program development techniques, allowing readers to choose methods that work best for them.

Multiple learning curves: To write a program, you have to understand the algorithm, know the programming language, and be able to debug errors. We discuss these and other aspects throughout the book and summarize our advice in Appendix ??.

Object-Oriented Programming

Some Java books introduce classes and objects immediately; others begin with procedural programming and transition to object-oriented more gradually.

Many of Java's object-oriented features are motivated by problems with previous languages, and their implementations are influenced by this history. Some of these features are hard to explain when people aren't familiar with the problems they solve.

We get to object-oriented programming as quickly as possible (beginning with Chapter ??). But we introduce concepts one at a time, as clearly as possible, in a way that allows readers to practice each idea in isolation before moving on. So it takes some time to get there.

You can't write Java programs (even Hello World) without encountering object-oriented features. In some cases we explain a feature briefly when it first appears, and then explain it more deeply later on.

If you read the entire book, you will see nearly every topic required for Java SE Programmer I certification. Supplemental lessons are available in the official Java tutorials on Oracle's website (<https://thinkjava.org/tutorial>).

This book is also well suited to prepare high school students for the AP Computer Science A exam, which includes object-oriented design and implementation. (AP is a registered trademark of The College Board.) A mapping of *Think Java* section numbers to the AP course is available on our website: <https://thinkjava.org/>.

Changes to the Second Edition

This new edition was written over several years, with feedback from dozens of instructors and hundreds of students. A complete history of all changes is available on GitHub. Here are some of the highlights:

Chapters 1–4: We reordered the material in Chapter 1 to present a more interesting balance of theory and practice. Chapters 2–3 are much cleaner now too. Methods are now presented in a single chapter, along with additional in-depth examples.

Chapters 5–8: We rearranged these chapters a lot, added many examples and new figures, and removed unnecessary details. Strings are covered earlier (before arrays) so that readers can apply them to loop problems. The material on recursion is now a chapter, and we added new sections to explain binary numbers and *CodingBat*.

Chapters 9–12: Our main goal for these chapters was to provide better explanations and more diagrams. Chapters 9–10 focus more on immutable versus mutable objects, and we added new sections on `BigInteger` and `StringBuilder`. The other content is largely the same, but it should be easier to understand now.

Chapters 13–17: We balanced the amount of content in Chapters 13–14 by moving `ArrayLists` earlier, and we implement the “War” card game as another example. Chapters 15–17 are brand new in this edition; they cover more advanced topics including 2D arrays, graphics, exceptions, abstract classes, interfaces, and events.

Appendixes: We added Appendix ?? to explain documentation comments and Javadoc in more detail. The other three appendixes that were present in the first edition have been revised for clarity and layout.

About the Appendixes

The chapters of this book are meant to be read in order, because each one builds on the previous one. We also include several appendixes with material that can be read at any time:

Appendix A, “Tools”

This appendix explains how to download and install Java so you can compile programs on your computer. It also provides a brief introduction to DrJava—an integrated development environment designed primarily for students—and other development tools, including Checkstyle for code quality and JUnit for testing.

Appendix B, “Javadoc”

It’s important to document your classes and methods so that other programmers (including yourself in the future) will know how to use them. This appendix explains how to read documentation, how to write documentation, and how to use the Javadoc tool.

Appendix C, “Graphics”

Java provides libraries for working with graphics and animation, and these topics can be engaging for students. The libraries require object-oriented features that students will not completely understand until after Chapter ??, but they can be used much earlier.

Appendix D, “Debugging”

We provide debugging suggestions throughout the book, but this appendix provides many more suggestions on how to debug your programs. We recommend that you review this appendix frequently as you work through the book.

Using the Code Examples

Most of the code examples in this book are available from a Git repository at <https://github.com/ChrisMayfield/ThinkJavaCode2>. Git is a “version control system” that allows you to keep track of the files that make up a project. A collection of files under Git’s control is called a “repository”.

GitHub is a hosting service that provides storage for Git repositories and a convenient web interface. It provides several ways to work with the code:

- You can create a copy of the repository on GitHub by clicking the **Fork** button. If you don't already have a GitHub account, you'll need to create one. After forking, you'll have your own repository on GitHub that you can use to keep track of code you write. Then you can "clone" the repository, which downloads a copy of the files to your computer.
- Alternatively, you could clone the original repository without forking. If you choose this option, you don't need a GitHub account, but you won't be able to save your changes on GitHub.
- If you don't want to use Git at all, you can download the code in a ZIP archive using the **Clone** button on the GitHub page, or this link: <https://thinkjava.org/code2zip>.

After you clone the repository or unzip the ZIP file, you should have a directory named *ThinkJavaCode2* with a subdirectory for each chapter in the book.

The examples in this book were developed and tested using OpenJDK 11. If you are using a more recent version, everything should still work. If you are using an older version, some of the examples might not.

Acknowledgments

Many people have sent corrections and suggestions over the years, and we appreciate their valuable feedback! This list begins with Version 4.0 of the open source edition, so it omits those who contributed to earlier versions:

- Ellen Hildreth used this book to teach Data Structures at Wellesley College and submitted a whole stack of corrections and suggestions.
- Tania Passfield pointed out that some glossaries had leftover terms that no longer appeared in the text.
- Elizabeth Wiethoff noticed that the series expansion of $\exp(-x^2)$ was wrong. She has also worked on a Ruby version of the book.

- Matt Crawford sent in a whole patch file full of corrections.
- Chi-Yu Li pointed out a typo and an error in one of the code examples.
- Doan Thanh Nam corrected an example.
- Muhammad Saied translated the book into Arabic and found several errors in the process.
- Marius Margowski found an inconsistency in a code example.
- Leslie Klein discovered another error in the series expansion of $\exp(-x^2)$, identified typos in card array figures, and helped clarify several exercises.
- Micah Lindstrom reported half a dozen typos and sent corrections.
- James Riely ported the textbook source from LaTeX to Sphinx.
<http://fpl.cs.depaul.edu/jriely/thinkapjava/>
- Peter Knaggs ported the book to C#.
<https://www.rigwit.co.uk/think/sharp/>
- Heidi Gentry-Kolen recorded several video lectures that follow the book.
<https://www.youtube.com/user/digipipeline>
- Waldo Ribeiro submitted a pull request that corrected a dozen typos.
- Michael Stewart made several suggestions for improving the first half of the book.
- Steven Richardson adapted the book for an online course and contributed many ideas for improving the text.
- Fazl Rahman provided detailed feedback, chapter by chapter, and offered many suggestions for improving the text.

We are especially grateful to the technical reviewers of the O'Reilly Media first edition: Blythe Samuels, David Wisneski, and Stephen Rose. They found errors, made many great suggestions, and helped make the book much better.

Likewise, we thank Marc Loy for his thorough review of the O'Reilly Media second edition. He contributed many corrections, insights, and clarifications.

Many students have given exceptional feedback, including Ian Staton, Tanner Wernecke, Jacob Green, Rasha Abuhantash, Nick Duncan, Kylie Davidson, Shirley Jiang, Elena Trafton, Jennifer Gregorio, and Azeem Mufti.

Other contributors who found one or more typos: Stijn Debrouwere, Guy Driesen, Andai Velican, Chris Kuszmaul, Daniel Kurikesu, Josh Donath, Rens Findhammer, Elisa Abedrapo, Yousef BaAfif, Bruce Hill, Matt Underwood, Isaac Sultan, Dan Rice, Robert Beard, Daniel Pierce, Michael Giftthaler, Chris Fox, Min Zeng, Markus Geuss, Mauricio Gonzalez, Enrico Sartirana, Kasem Satitwiwat, Jason Miller, Kevin Molloy, Cory Culbertson, Will Crawford, Shawn Brenneman, Tom Nguyen, and Ala-Uddin Mohammad.

If you have additional comments or ideas about the text, please send them to: feedback@greenteapress.com.

Allen Downey and Chris Mayfield

